# BioSpecInfo

## Deploy Guide — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v232** · commit `e041ec9`

Generato il 2026-10-23

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

# Deploy Guide — BioSpecInfo

| Field | Value |
|---|---|
| **Software** | BioSpecInfo |
| **Version described** | `bsi-v232` |
| **Purpose** | Operational procedure for publishing, updating and rolling back the application. |

## 0. What gets published, in practice

BioSpecInfo is a set of **static files**. There is nothing to compile, no process to keep running, no database to provision.

| | |
|---|---|
| **Build** | None. The repository files *are* the application. |
| **Server runtime** | None |
| **Environment variables** | None for the application. Only the optional proxy has any (§4). |
| **Database** | None server-side |
| **Server requirement** | Serve static files over HTTPS with the correct MIME types |

The one non-obvious requirement: the server must return `application/wasm` for `.wasm` files, otherwise RDKit and SQLite do not start. GitHub Pages, Netlify, Vercel and Cloudflare Pages already do.

## 1. Environments

| Environment | How it is obtained | What it is for |
|---|---|---|
| **Development** | `python3 -m http.server 8899` from the root | Daily work and verification benches |
| **Preview** | A branch published on Pages, or `npx serve` | Checking before merging |
| **Production** | GitHub Pages from the `main` branch | `samupropio1-ship-it.github.io/BioSpecInfo-v11/` |

> **Why a server is needed even in development.** Opening `index.html` with a double click ( `file://` ) does not work: RDKit and SQLite are WebAssembly modules and require an HTTP context. It is the reason the verification benches also run against a local server.

# 2. Publishing to production

## 2.1 Procedure

```
# 1. from the working branch, the full battery
python3 -m http.server 8899 &
node tools/genera-evidenza.js          # must close with 0 failures

# 2. bump the version — with the tool, NEVER by hand
node tools/porta-versione.js bsi-v<NNN+1> --prova   # look first
node tools/porta-versione.js bsi-v<NNN+1>           # then write

# 3. regenerate SBOM, evidence, PDFs and packages — IN THIS ORDER
#    The packages bundle the PDFs and the evidence: built first, they carry
#    the typeset documents and the report of the PREVIOUS version.
#    genera-pacchetti now notices and stops, but better not to get there.
node tools/genera-sbom.js
node tools/genera-evidenza.js       # the battery; needs a server on :8899
node tools/genera-pdf.js            # 36 PDFs; every footer carries version and commit
node tools/genera-pacchetti.js      # the three packages; fails on broken links, stale PDFs or
stale evidence

# 4. merge into main
git add -A && git commit -m "…"
git push -u origin <branch>
#    then pull request and squash merge
```

GitHub Pages publishes automatically from `main`: the *pages build and deployment* workflow takes about a minute.

## 2.1-bis Publishing the version's release

The application is online as soon as `main` updates; the **release** serves a different purpose: giving an assessor a fixed point to download — a tag, the version notes and the delivery packages attached.

It is published from the repository, not from outside:

> **Actions → release → Run workflow**, field `versione`: `bsi-v<NNN>`.

The workflow creates the tag on the current commit, opens the release and attaches the three packages from `consegna/`. The full archive — `models/` included, about 124 MB — is generated by GitHub itself as *Source code (zip)*.

**Why from the repository and not from the command line.** The credentials a remote session uses on the repository are authorised to write to branches and **not to tags**: the `POST` to `git-receive-pack` for `refs/tags/…` answers `403 Forbidden`, while a new branch goes through. The Actions token, by contrast, writes to the repository that hosts it.

What the workflow **refuses** to do:

| Condition | Outcome |
|---|---|
| `CHANGELOG.md` has no `## [bsi-v<NNN>]` entry with content | fails: a release without notes looks like a version and does not say what changed |
| fewer than three packages in `consegna/` | fails: a missing attachment would make the release silently incomplete |
| the tag does not point at the commit the packages came from | fails: a release must have a single provenance |

That last row exists because of a real defect. On re-running the workflow to refresh the packages, `gh release edit` updated notes and assets but **did not move the tag**: the *Source code (zip)* archive, which GitHub generates from the tag, stayed at the first publication's commit while the packages came from a later one. Two provenances, and nothing saying so. The tag is now moved, and a following step does not take that on trust: it asks GitHub where the tag points and compares.

The workflow is **re-runnable**: if the release already exists it updates notes, assets and tag instead of failing halfway. Moving an already-published tag is not free — whoever downloaded it already has different content — so it applies to a release just published, not an old one: for that, publish a new version.

## 2.1-ter Why the version bump is not done with a find-and-replace

It looks like the most harmless operation in the world: search `bsi-v<NNN>`, write `bsi-v<NNN+1>`. **For three releases in a row it was done that way, and every time it rewrote historical sentences** — sentences recording a fact that happened at a past version. What came out of it sat inside a signed conformance statement:

| the document said | the truth |
|---|---|
| the password had been **changed at the current version** | it was changed at version `bsi-v188` |
| the tag had been **moved to the current version's commit** | the incident happened at version `bsi-v188` |
| the thirty-six entries were **added in the current version** | they were added in `bsi-v188` |
| "bump `CACHE` (e.g. `bsi-vN` → `bsi-vN`)" | an example equal to itself |

No check could catch it: every line named a version that **exists**, in the document describing that version.

> This table uses placeholders rather than the real numbers because the two checks described below **would reject it** — which is the proof that they work.

`tools/porta-versione.js` touches **only** the lines that DECLARE the version — headers, footers, the README badge, package names, the two code lines — and **lists** everything else so it gets read. With `--prova` it writes nothing.

Two checks in `tools/verifica-documenti.js` now close the door: no past-tense sentence may name the current version, and no upgrade example may show the same version on both sides.

> **Name the thing, not the number.** "From the Pretsch tables" rather than "from version such-and-such": it reads better and does not age.

### 2.2 The two version lines

`CACHE` in `sw.js` and `BSI_APP_VERSION` in `index.html` **must match**.

- `CACHE` determines whether the Service Worker considers the cache stale:

without a bump, devices keep serving the old files and the publication reaches nobody.

- `BSI_APP_VERSION` is what the user sees in the "Updates" window and what the

update check compares with the `sw.js` on GitHub.

They drifted apart once in the past — the second had been stuck for months — and the result was that the update check was comparing the wrong number.

### 2.3 How the update reaches users

1. The browser detects a different `sw.js` and installs the new Service Worker.
2. `skipWaiting()` activates it at once; the old caches are deleted.
3. The Service Worker **notifies** the open tabs ( `BSI_SW_UPDATED` ) — it does not

reload them on its own authority.

1. The page decides: if no work is in progress it reloads itself; if the

assistant is answering or there is unsaved text, it shows "Update now / Later".

Whoever opens the app at that moment already receives the new version.

---

## 3. Publishing alternatives

The application does not depend on GitHub Pages. Any static file host will do, provided it is over HTTPS (the Service Worker requires it).

| Platform | Command / configuration | Notes |
|---|---|---|
| **GitHub Pages** | Settings → Pages → branch `main`, folder `/` | In use. Free, custom domain possible |
| **Netlify** | `netlify deploy --prod --dir=.` | No build command |
| **Vercel** | `vercel --prod` | Static project, no framework |
| **Cloudflare Pages** | `npx wrangler pages deploy .` | Convenient if the proxy is already in use |
| **VPS with nginx** | Copy the files to `/var/www/biospecinfo` | See the configuration below |

**nginx configuration**

```
server {
    listen 443 ssl http2;
    server_name biospecinfo.example.org;
    root /var/www/biospecinfo;

    # Without this MIME type, RDKit and SQLite do not start
    types { application/wasm wasm; }

    # sw.js must not be cached by the CDN: it is what announces new versions
    location = /sw.js {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

### A rule that holds everywhere: `sw.js` must not be cached

If a CDN keeps `sw.js`, the browser keeps seeing the old version and **no update ever arrives**, however much one publishes. This holds on every platform.

---

## 4. Optional proxy — a separate deployment

The proxy is a **distinct** component, owned by the user, published on their own Cloudflare account. It is not required in order to use the application.

```
cd proxy
npx wrangler login
npx wrangler deploy
# the address is printed: https://spectra-proxy.<account>.workers.dev

npx wrangler secret put GROQ_KEYS      # one or more keys, comma-separated
npx wrangler secret put GEMINI_KEYS    # optional
```

The address is then pasted into the assistant's "Proxy" box, inside the app. The same four steps are available **inside the application**, with the commands ready to copy.

### Recommended configuration

```
# Without this variable the proxy accepts requests from any origin
npx wrangler secret put ORIGINI
# value: https://samupropio1-ship-it.github.io
```

| Variable | Default | Why change it |
|---|---|---|
| `ORIGINI` | *(empty = all)* | **Always set it.** An open proxy is usable by anyone, at the expense of your quota. |
| `LIMITE_IP` | `20` | Requests per IP within the window |

| Variable | Default | Why change it |
|---|---|---|
| `TETTO_GIORNO` | `2000` | Overall daily ceiling |

> **Keys must never go into `wrangler.toml`.** That file is versioned: a key written there ends up on GitHub, and the bots that scan it find it within hours. Use `wrangler secret put`, which keeps them out of the repository.

---

## 5. Domain and certificate

| Aspect | Situation |
|---|---|
| **HTTPS** | Mandatory: without it the Service Worker does not register and the app does not work offline |
| **Certificate** | Provided and renewed automatically by GitHub Pages |
| **Custom domain** | Settings → Pages → Custom domain, plus a `CNAME` record pointing to `<user>.github.io` |

---

## 6. Monitoring

There is no server to monitor: there are no processes that could fall over, nor resources that could run out.

| What to watch | How |
|---|---|
| Outcome of the publication | GitHub → Actions → *pages build and deployment* |
| Availability | An HTTP check on `index.html` (UptimeRobot or similar) |
| Client-side errors | The browser console. Remote telemetry exists but is **disabled** (`BSI_TELEMETRY_URL` empty): enabling it requires a deliberate decision about data handling. |
| Proxy consumption | Cloudflare → Workers → Analytics |

---

## 7. Rollback and emergency procedures

### 7.1 Going back to the previous version

```
git revert <commit>
git push origin main
```

Pages republishes in about a minute. Devices receive the restored version as a normal update — **provided `CACHE` changes**: a revert that also takes the version number backwards is fine, because what counts is that the value be *different*, not that it be greater.

### 7.2 "I published but I do not see the changes"

In order:

1. **Was the version bumped?** It is the cause in ninety per cent of cases.

Without it, the Service Worker serves the cache.

1. **Has Pages finished?** GitHub → Actions.
2. **Is `sw.js` cached by a CDN?** It must answer `no-cache`.
3. From the app: ✨ menu → **Updates** → *Check*, which forces a

`registration.update()`.

1. As a last resort: developer tools → Application → Service Workers →

*Unregister*, then reload.

### 7.3 "The assistant has stopped answering"

This is not a publishing problem: it concerns the AI provider. In the app, the 🔌 **Test** button measures in a few seconds which providers the device can actually reach, without needing any key.

### 7.4 Restoring the user's data

The data live on the device and **there are no copies on a server**: they cannot be restored remotely. This is a direct consequence of the local-first architecture, and it is declared to the user. Exports (notes, documents) remain the only form of backup.

---

## 8. Pre-publication checklist

- ☐ `node tools/genera-evidenza.js` closes with **0 failures**
- ☐ `CACHE` in `sw.js` and `BSI_APP_VERSION` in `index.html` bumped and **equal**
- ☐ `node tools/genera-sbom.js` re-run
- ☐ `node tools/genera-pdf.js` re-run — otherwise the attachments describe the previous version
- ☐ `node tools/genera-pacchetti.js` re-run and closed with **0 broken links**
- ☐ Evidence report regenerated against the commit being published
- ☐ Any new deviations noted in `09-Release-Conformance-Statement.md` §4
- ☐ No conflict markers (`verifica_guida` checks this across 58 files)
- ☐ No API key in the versioned files
- ☐ The accessibility debt **has not grown** — if it has fallen, regenerate the

baseline with `node tools/verifica-accessibilita.js --aggiorna-riferimento` and record it in the commit

- ☐ Code coverage **has not fallen** — `audit_copertura` fails if it has

---

_Document updated to version `bsi-v232`._